

Password Protection Lab (CodeGrade)

 <https://github.com/learn-co-curriculum/python-p4-passwords-lab>  <https://github.com/learn-co-curriculum/python-p4-passwords-lab/issues/new>

Learning Goals

- Implement login and signup with a password.
- Use **Bcrypt** to salt and hash passwords.

Key Vocab

- **Identity and Access Management (IAM)**: a subfield of software engineering that focuses on users, their attributes, their login information, and the resources that they are allowed to access.
- **Authentication**: proving one's identity to an application in order to access protected information; logging in.
- **Authorization**: allowing or disallowing access to resources based on a user's attributes.
- **Session**: the time between a user logging in and logging out of a web application.
- **Cookie**: data from a web application that is stored by the browser. The application can retrieve this data during subsequent sessions.

Introduction

We're going to make a Flask app that covers a simple authentication flow: users can create accounts, log in, and log out.

There is some starter code in place for a Flask API backend and a React frontend. To get set up, run:

```
$ pipenv install && pipenv shell  
$ npm install --prefix client  
$ cd server  
$ flask db upgrade
```

You can work on this lab by running the tests with `pytest -x`. It will also be helpful to see what's happening during the request/response cycle by running the app in the browser. You can run the Flask server with:

```
$ python app.py
```

And you can run React in another terminal with:

```
$ npm start --prefix client
```

You don't have to make any changes to the React code to get this lab working.

Configuration

Before we begin, take note of the new `config.py` file in the `server/` directory. Flask is a fun framework in that it gives us many different ways to structure our applications- occasionally though, we're forced to adopt a new architecture.

In each of our applications so far, `app.py` needed to import from `models.py` in order to initialize the database's connection to the app. That's still the case here, but we also find ourselves with the need to import an instantiated `Bcrypt` from `app.py` into `models.py`! This creates a *circular import*, where objects in two separate files are dependent upon one another to function.

To avoid this, you can often refactor your objects to avoid unnecessary dependencies (we're all guilty of this!), you can refactor your code into one large file, or you can move some of your imports and configurations into a third file. That's what we did here- check out `config.py` and you'll notice a lot of familiar code. We took the imports and configurations from `app.py` and `models.py` and put them together to avoid circular imports. These are then imported by `app.py` and `models.py` when they're ready to be used.

Setup

Our app has three pages:

1. A signup page, where the user enters their username, password, and password confirmation.
2. A login page, where the user submits their username and password and are then logged in.
3. A user homepage, which says, "Welcome, \${username}!"

Users should not be able to log in if they enter an incorrect password.

Note: we're not covering password validations in this lab, so don't worry about those. Password validation is hard to get right anyway — it's surprisingly easy to produce rules that decrease password security rather than enhance it.

Instructions

To complete the lab and get the tests passing, you will need to:

- Create a `Signup` resource with a `post()` method that responds to a `POST /signup` request. It should: create a new user; save their hashed password in the database; save the user's ID in the session object; and return the user object in the JSON response.
- Add a `get()` method to your `CheckSession` resource that responds to a `GET /check_session` request. If the user is authenticated, return the user object in the JSON response. Otherwise, return an empty response with a 204 status code.
- Create a `Login` resource with a `post()` method for logging in that responds to a `POST /login` request and returns the user as JSON.
- Create a `Logout` resource with a `delete()` method for logging out that responds to a `DELETE /logout` request.

Happy coding!

Resources

- **Flask-Bcrypt**  (<https://flask-bcrypt.readthedocs.io/en/1.0.1/>)

This tool needs to be loaded in a new browser window

Load Password Protection Lab (CodeGrade) in a new window